

Navegação Autónoma na Formula Student: Uma Abordagem Baseada em Aprendizagem por Reforço

Luís Miguel Pereira Silva and Pedro Miguel S. A. Urbano dos Reis

Mestrado em Inteligência Artificial, Universidade do Minho, Braga, Portugal
{pg60390,pg59908}@alunos.uminho.pt

Abstract. Este relatório apresenta um sistema de condução autónoma para a categoria *Driverless* da Formula Student, treinado de raiz com aprendizagem por reforço. Foi desenvolvido um ambiente de simulação 2D, compatível com a API *Gymnasium*, que integra um modelo cinemático de bicicleta, um sensor de cones com ruído realista, 14 traçados oficiais de competição em formato YAML e as regras FS-AI de penalização por cone, *Down-Or-Out* e *off-course* gradual. O agente principal é um *Soft Actor-Critic*, comparado com *Proximal Policy Optimization* como *baseline*, treinado com *domain randomization* e avaliado sob um *split* canónico de 9 pistas de treino e 5 inéditas de teste. O SAC apresenta um *lap rate* comparável ao PPO (66% vs. 68%) mas é cerca de duas vezes mais seguro (2.4 vs. 4.8 cones derrubados). Identificámos, via pistas espelhadas, um viés direcional sistemático (−25 pp, invisível ao *split* de teste tradicional) e uma assimetria de robustez de perceção: a política tolera falhas transientes do sensor mas colapsa com cones fisicamente em falta. Ambas as fraquezas se corrigem com duas *data augmentations* dirigidas, *mirror* e *cone-dropout*, cuja combinação é super-aditiva e sem custo no desempenho base. Quinze ablações e um estudo iterativo de *reward shaping* quantificam a contribuição de cada componente.

Keywords: Aprendizagem por Reforço · Condução Autónoma · Formula Student · Soft Actor-Critic · Domain Randomization

1 Introdução e Definição do Problema

A Formula Student é a maior competição de engenharia automóvel universitária a nível mundial. A categoria *Driverless* desafia as equipas a desenvolver veículos totalmente autónomos, capazes de navegar numa pista delimitada por cones coloridos — azuis na fronteira esquerda, amarelos na direita e laranja na linha de partida/chegada — sem qualquer intervenção humana [1]. O veículo deve detetar cones, planear a trajetória e executar comandos de direção e aceleração em tempo real, completando a volta o mais depressa possível e sem derrubar cones; cada cone derrubado acrescenta uma penalização de tempo.

A abordagem clássica decompõe o problema num *pipeline* de perceção, mapeamento, planeamento e controlo. Neste trabalho exploramos uma alternativa

end-to-end baseada em aprendizagem por reforço [2]: treinar um agente que aprenda a conduzir diretamente da sua percepção local, maximizando o progresso ao longo da pista e minimizando penalizações. O agente não tem acesso ao traçado global nem à *centerline* ideal — toda a informação de pista provém da observação dos cones, replicando o cenário *onboard* real.

Estado da arte. Os sistemas de competição estabelecidos, como o AMZ Driverless [1], seguem o *pipeline* modular clássico de SLAM com cones, planeamento e controlo MPC, que exige engenharia extensa de cada módulo. Em paralelo, o RL *end-to-end* tem produzido políticas competitivas em simuladores de corrida, predominantemente com SAC [4] e PPO [5], com implementações de referência no Stable-Baselines3 [6], incluindo aplicações recentes à Formula Student Driverless [8]; a transferência para fora da distribuição de treino apoia-se em *domain randomization* [7] e *data augmentation*. A literatura avalia quase sempre em pistas inéditas, mas raramente sonda eixos de generalização estruturais, como a direção das curvas ou a densidade de cones — o foco da nossa avaliação.

Este relatório documenta a solução implementada e os resultados obtidos. As contribuições principais são: (i) um ambiente de simulação FS-AI 2D, leve e reproduzível, com 14 pistas reais de competição; (ii) uma comparação *multi-seed* entre SAC e PPO sob um protocolo de generalização rigoroso; (iii) a identificação e correção de um viés direcional estrutural, normalmente mascarado pelos *splits* de teste convencionais; (iv) uma análise de robustez de percepção que distingue falhas transitórias de cones em falta, corrigida com *augmentation* dirigida; e (v) um conjunto de 15 ablações e um estudo iterativo de *reward shaping* que isolam o contributo de cada decisão de *design*. O código encontra-se em <https://github.com/pedroreis2468/AR2>; o notebook de avaliação executável, `notebooks/overview.ipynb`, reproduz todas as tabelas e figuras deste relatório.

2 Ambiente de Simulação

O treino é conduzido inteiramente em simulação, num ambiente desenvolvido de raiz, `FSRacingEnv`, que segue a API *Gymnasium*. A simulação opera a 50 Hz com *action repeat* de 2, pelo que o agente decide a 25 Hz.

Modelo do veículo. Utilizámos um modelo cinemático de bicicleta [3] referenciado ao centro de gravidade, calibrado para um carro FS típico: ~ 250 kg, *wheelbase* de 1.55 m e $\mu = 1.5$. O modelo inclui *rate limiting* no *steering*, com $\dot{\delta}_{\max} = 1.5$ rad/s, e uma restrição de *friction circle*, $a_{\text{lat}}^2 + a_{\text{lon}}^2 \leq (\mu g)^2$, que limita a aceleração combinada e penaliza implicitamente o excesso de velocidade em curva.

Pistas: reais e procedurais. O sistema suporta duas fontes de pistas. (i) Pistas reais de competição: 14 traçados oficiais FS em formato YAML, obtidos do projeto PacSim — FSG19/21/23/24, FSE22 em duas variantes, FSE23/24, FSI24, FSCZ24, FSO20, FSS19 e FSS22 V1/V2 — carregados por um `YAMLTrackLoader` e suavizados com *splines* cúbicas periódicas. (ii) Geração procedural: pontos

de controlo numa elipse perturbada, suavizados com *splines* fechadas e re-amostrados a cada episódio — usados como *fallback*.

Percepção. Um sensor de cones com FOV de 180° e alcance de 15 m devolve os 3 cones mais próximos de cada cor no referencial do carro, com ruído gaussiano de $\sigma = 0.2$ m na distância e $\sigma = 0.007$ rad no *bearing*, e probabilidade de deteção de 95%, emulando uma *pipeline* de visão YOLO + *stereo depth*. Cones derrubados são removidos persistentemente do FOV, refletindo o seu impacto na percepção real.

Regras FS-AI e colisão. A deteção de colisão usa uma *bounding box* retangular do veículo. Cada cone azul/amarelo derrubado custa +2s de penalização de tempo e os cones laranja custam o dobro. A terminação do episódio por *Down-Or-Out* ocorre quando: (a) ≥ 10 cones são derrubados; (b) o desvio lateral à *centerline* excede 5 m; ou (c) o veículo permanece *off-course* mais de 2s consecutivos. Detetam-se também contramão e estagnação — progresso inferior a 2 m em 300 *steps*.

3 Metodologia

O problema é formulado como um processo de decisão de Markov com $\gamma = 0.99$.

Observação ($\mathbf{s} \in \mathbb{R}^{24}$). 6 dimensões de estado do veículo — v_x , v_y , ω , δ , a_x e a_y , normalizadas — e 18 dimensões com as posições relativas (x, y) dos 3 cones mais próximos de cada cor: azul, amarelo e laranja. Sublinhe-se que o agente *não* recebe a *centerline* nem as distâncias às fronteiras: o planeamento global tem de emergir do comportamento reativo.

Ação ($\mathbf{a} \in [-1, 1]^2$). *Steering* normalizado e um comando unificado de *throttle*/travão, em que valores negativos travam.

Reward shaping. A recompensa por *step* combina os termos da Tabela 1. O termo de progresso, com sinal ao longo da *centerline* ideal, é o sinal dominante; os restantes moldam um comportamento rápido, alinhado, suave e dentro da pista. Penalizações terminais de -50 para DOO, -30 para *off-course timeout* e -20 para contramão, juntamente com um bônus de volta de $\max(200 - 5n_{\text{cones}}, 50)$, calibram os comportamentos-limite. Esta configuração final resultou de um processo iterativo, documentado como estudo de *design* na Secção 5.1.

Algoritmo principal: SAC. O *Soft Actor-Critic* [4] é um algoritmo *off-policy* com maximização de entropia, adequado a controlo contínuo. Implementámos uma versão *custom* em PyTorch — *Gaussian actor squashed*, *twin Q-networks* MLP 256×256 , *target networks* com *Polyak* $\tau=0.005$, *replay buffer* de 1 M e *auto-tuning* da temperatura α — para fins educativos, e usámos a implementação de produção do Stable-Baselines3 [6] para os resultados reportados. A *target entropy* foi fixada em -0.5 , em vez do *default* $-\dim(\mathcal{A}) = -2$, para evitar o colapso prematuro da exploração.

Table 1. Componentes da função de recompensa.

Componente	Descrição	Peso
r_{progress}	Distância percorrida na <i>centerline</i> (com sinal)	$\times 2.0$
$r_{\text{alignment}}$	$v \cdot \cos(\theta_{\text{err}})$ — rápido e alinhado	$\times 0.4$
r_{smooth}	Penaliza $\Delta \text{steering}$ acima de <i>dead-band</i> 0.1	$\times 0.05$
r_{lateral}	Linear dentro da pista, quadrática fora	$\times 0.08$
r_{time}	Custo fixo por <i>step</i> (encoraja velocidade)	-0.005
<i>cone hit</i>	Por cone azul/amarelo derrubado	-8.0
<i>orange hit</i>	Por cone laranja derrubado	-16.0
<i>lap bonus</i>	Por volta completa, descontado pelos cones	$+200$

Baseline: PPO. Como *baseline on-policy* usámos a implementação SB3 do PPO [5], com a mesma arquitetura de rede 256×256 , para uma comparação justa.

Domain randomization. Para promover robustez, cada episódio sorteia massa ($\pm 15\%$), atrito ($\pm 20\%$) e arrasto ($\pm 10\%$) do veículo, posição lateral e velocidade iniciais, e o ruído do sensor [7,8].

Data augmentation dirigida. Em resposta às fraquezas estruturais identificadas nas Secções 5.5 e 5.6, introduzimos duas técnicas de *augmentation* ao nível dos dados, sem alterar a função de recompensa: (i) *mirror augmentation* — com probabilidade 50% a cada *reset*, espelha a pista em Y, trocando cones azuis↔amarelos e recalculando tangentes e normais; (ii) *cone-dropout augmentation* — com probabilidade 50% a cada *reset*, remove uma fração uniforme em $[0, 15\%]$ dos cones físicos, de forma persistente no episódio, mantendo intactas a *centerline* e, portanto, a recompensa.

4 Implementação e Protocolo Experimental

Implementação e arquitetura de software. O sistema, em Python/PyTorch, organiza-se em quatro camadas: `env/`, com o ambiente `FSRacingEnv` e os componentes de veículo, sensor, *loader* YAML, gerador procedural e *renderer* PyGame; `agent/`, com o SAC *custom* educativo; `train.py` e `evaluate.py`, para treino SB3 com 4–8 ambientes paralelos e avaliação com visualização; e `scripts/`, para avaliação per-pista, agregação multi-*seed*, figuras, espelhamento e *sweeps* de robustez. Uma suite `pytest` com 16 testes cobre o contrato Gymnasium, o veículo e os *splits*. Treino numa GPU RTX 5070 Ti; avaliação em CPU.

Splits canónicos. As 14 pistas de circuito são divididas num *split* fixo de 9 pistas de treino — FSG19, FSG21, FSG23, FSE22, FSE22_test, FSE23, FSO20, FSS19 e FSS22_V1 — e 5 pistas de teste mantidas completamente inéditas durante o treino: FSG24, FSI24, FSCZ24, FSE24 e FSS22_V2. Como terceiro eixo de avaliação, geram-se as 14 versões espelhadas em Y, mantidas *held-out*, que preservam a geometria mas invertem o sentido das curvas.

Table 2. Estudo de design da recompensa: 6 iterações, cada uma treinada até à convergência (SAC, 500k *steps* $\times$ 3 *seeds*). Métricas na pista de avaliação, $\bar{x} \pm \sigma$ entre *seeds*.

# Mudança na recompensa	Pista (%)	Vel. (km/h)	Cones
1 v0 “suicide policy”: <i>off-track</i> = morte instantânea	76 ± 23	60 ± 22	2.7
2 <i>Off-course</i> gradual (substitui a morte instantânea)	83 ± 30	49 ± 26	3.4
3 Suavidade $-0.3 \Delta\delta \rightarrow -0.05$ com <i>dead-band</i> 0.1	100 ± 0	74 ± 7	4.0
4 Lateral quad. $\times 0.2 \rightarrow$ <i>piecewise</i> $\times 0.08$	100 ± 0	62 ± 22	2.2
5 Reescala (progr. $\times 2.0$, align $\times 0.4$, tempo -0.005)	99 ± 1	79 ± 3	1.6
6 + anti-estagnação (<i>checkpoint</i> DOO) = final	100 ± 0	83 ± 1	1.8

Modelos treinados. SAC *baseline*, com 3M *steps* e 3 *seeds*; PPO *baseline*, com 6M *steps* e 3 *seeds*; 15 ablações de SAC, com 2M *steps* e 1 *seed* cada; e o *design* factorial de *augmentation* — SAC com *mirror*, com *cone-dropout* e com ambas, a 3M *steps* e 1 *seed* cada.

Métricas. *Lap rate*, a percentagem de episódios em que a volta é completada; número médio de cones derrubados; velocidade média; e tempo de volta — agregados como $\bar{x} \pm \sigma$ entre *seeds*.

Nota metodológica: avaliação estocástica. A avaliação per-track corre N=10 episódios por pista com *domain randomization* ativa: os valores são *estimativas da performance média sob variação*, não medições determinísticas, e re-execuções variam $\approx \pm 5\text{--}8$ pp no *lap rate*; as fontes de aleatoriedade usam o gerador global do NumPy, não semeado pelo avaliador. Diferenças de poucos pontos percentuais estão portanto dentro do ruído amostral; os efeitos grandes — consistência SAC vs. PPO, viés direcional, efeito das *augmentations* — situam-se muito acima dele.

5 Resultados e Discussão

5.1 Engenharia iterativa da recompensa

Antes de qualquer comparação de algoritmos, o maior desafio prático foi obter uma recompensa que produzisse *algum* comportamento útil: nas primeiras versões o carro mal se mexia ou não arriscava as curvas. Cada iteração foi treinada até à convergência, com SAC a 500k *steps* $\times$ 3 *seeds*. A Tabela 2 resume a progressão e a Figura 1 a sua evolução empírica.

A leitura central: com orçamento suficiente, quase todos os estágios completam a pista — o que distingue as recompensas é a *qualidade* e a *fiabilidade*. As recompensas das iterações 1–2 são uma lotaria entre *seeds*, com desvios de 20–30 pp na completção e 20–26 km/h na velocidade; o *dead-band* da iteração 3 torna o treino fiável, com 100% de completção sem variância; o lateral *piecewise* e a reescala das iterações 4–5 reduzem os cones de ~ 4 para $\sim 1.6\text{--}1.8$; e a

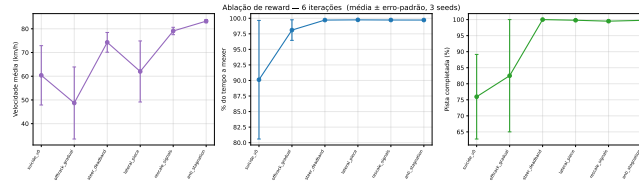


Fig. 1. Evolução empírica ao longo das 6 iterações de recompensa (SAC à convergência, $500k \text{ steps} \times 3 \text{ seeds}$, média $\pm$ erro-padrão). Gerada por `scripts/reward_iterations.py`.

Table 3. Resumo no *test split* FS-AI (5 pistas inéditas, 10 episódios/pista, política determinística, *domain randomization* ativa). Valores: $\bar{x} \pm \sigma$ entre seeds.

Modelo	Seeds	Voltas (%)	Cones	Vel. (km/h)
SAC	3	66 ± 0	2.4 ± 0.6	76 ± 1
PPO	3	68 ± 7	4.8 ± 0.9	79 ± 1
SAC+Mirror Aug.	1	72	2.9	67

recompensa final é a mais rápida, a 83 ± 1 km/h. O *reward shaping* deixou de decidir *se* o carro conduz e passou a decidir *quão bem e quão fiavelmente*. Este estudo é o complemento aditivo do estudo de ablações da Secção 5.4, que mede o efeito *subtrativo* de remover componentes da recompensa final.

5.2 SAC vs. PPO: o *trade-off* velocidade–segurança

A Tabela 3 resume o desempenho no *test split*. SAC e PPO atingem *lap rates* estatisticamente indistinguíveis (66% vs. 68%, dentro do ruído amostral), mas o PPO derruba em média o dobro dos cones: 4.8 contra 2.4. O SAC é portanto uma política claramente mais conservadora e segura, enquanto o PPO é marginalmente mais rápido, 79 contra 76 km/h, à custa de agressividade.

A análise per-pista — tabelas completas no repositório, em `results/aggregated/` — revela que a dificuldade é fortemente dependente do traçado: ambas as políticas completam quase sempre FSI24, FSCZ24 e FSS22_V2, com 93–100%, mas falham consistentemente em FSE24, com apenas 7–17% de *lap rate*, e em FSG24, com 30–33% — as pistas com as curvas mais apertadas. É também em FSE24 que o PPO mais derruba cones, 10.7 em média contra 4.5 do SAC, confirmando o seu perfil agressivo nas situações difíceis.

5.3 Curvas de aprendizagem

A Figura 2 mostra a recompensa de avaliação ao longo do treino, agregada sobre as 3 *seeds*. O SAC, *off-policy*, é substancialmente mais eficiente em amostras:

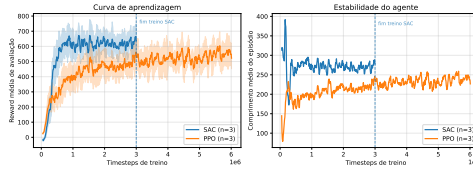


Fig. 2. Curvas de aprendizagem (recompensa de avaliação vs. *steps*), média $\pm\sigma$ sobre 3 *seeds*, para SAC e PPO.

atinge o seu plateau com cerca de 3M *steps*, enquanto o PPO necessita de aproximadamente o dobro, 6M, para convergir para um desempenho comparável — esperado, dado o reaproveitamento de transições do *replay buffer*.

5.4 Ablações

A Tabela 4 quantifica o efeito de remover ou alterar cada componente, relativamente a um *baseline* de SAC mais curto, treinado com 2M *steps* e 1 *seed*, com *lap rate* de 60%. Atenção à leitura: o *lap rate* isolado é enganador, porque algumas variantes “ganham” voltas à custa de derrubar muito mais cones. Os casos mais ilustrativos são as ablações de sensor: reduzir o alcance para 8m sobre o *lap rate* para 98%, mas o veículo passa a arrastar-se a 52 km/h e a derrubar 16 cones por episódio — aprende a “empurrar” os cones em vez de os evitar. O mesmo padrão — mais voltas, muito mais cones — ocorre com FOV de 90° e 360°, confirmando que o FOV de 180° tem um *sweet spot* percepção–desempenho *não monotónico*.

Quanto à *domain randomization*, removê-la melhora ligeiramente o *lap rate* de teste, em +8 pp, mas o seu valor real é a robustez fora-de-distribuição — *sim-to-real* e variação de condições — que o *test split* de pistas originais não captura. As ablações de hiperparâmetros — LR, *buffer*, *batch*, *target entropy* — produzem variações que, na sua maioria, caem dentro do ruído amostral de ± 5 –8 pp, reforçando a robustez da configuração escolhida.

5.5 Viés direcional e *mirror augmentation*

A descoberta mais relevante deste trabalho surge ao avaliar os modelos em pistas espelhadas em Y: mesma geometria, sentido das curvas invertido. A Tabela 5 mostra uma quebra sistemática do *lap rate*: −26 pp no SAC (66%→40%) e −23 pp no PPO (68%→45%). O SAC exhibe um padrão bimodal, de 10% ou 100% por pista, enquanto o PPO degrada uniformemente. Isto indica que as políticas sobre-ajustaram a estrutura direcional do conjunto de treino — um *overfit* que o *test split* FS-AI tradicional não revela, por partilhar essa mesma estrutura.

Treinando uma *seed* de SAC com *mirror augmentation*, o viés desaparece: o modelo aumentado atinge 74% em pistas espelhadas, contra 40% do *baseline* — um ganho de 34 pp — e *melhora* também nas originais, para 72%, mantendo a

Table 4. Ablações (SAC, 2M steps, 1 seed). Lap rate, cones e Δ face ao *baseline* (60%, 3.2 cones), médios sobre as 5 pistas de teste.

Categoria	Variante	Voltas (%)	ΔV (pp)	Cones	ΔC
Baseline	Default (<i>target entropy</i> -0.5)	60	—	3.2	—
Recompensa	Sem termo de <i>alignment</i>	68	+8	2.7	-0.5
	Sem <i>persistent knockdown</i>	66	+6	3.1	-0.1
Domain Rand.	Sem <i>domain randomization</i>	68	+8	4.5	+1.3
Sensor	Sem ruído de sensor	76	+16	4.9	+1.7
	FOV 90°	84	+24	9.2	+6.0
	FOV 360°	84	+24	17.4	+14.2
	Alcance 8 m	98	+38	16.3	+13.1
Dinâmica	Vel. máx. 60 km/h	52	-8	3.5	+0.3
Observação	<i>Legacy obs</i> (18 dims)	64	+4	3.1	-0.1
Hiperparams	LR 10^{-4}	68	+8	4.9	+1.7
	<i>Buffer</i> 10^6	68	+8	3.2	+0.0
	<i>Batch</i> 512	74	+14	3.2	-0.0
	<i>Target entropy</i> -1.0	60	+0	2.8	-0.4
	<i>Target entropy</i> -2.0	72	+12	3.0	-0.2

Table 5. Viés direcional: *lap rate* em pistas originais (*test*) vs. espelhadas em Y (*mirror*, *held-out*). $\bar{x} \pm \sigma$ entre seeds.

Modelo	Test (%)	Mirror (%)	Δ (pp)
SAC	66 ± 0	40 ± 11	-26
PPO	68 ± 7	45 ± 6	-23
SAC+Mirror Aug.	72	74	+2

segurança com 2.9 cones. O problema era portanto estrutural ao protocolo de treino e não ao algoritmo, e é corrigível com uma intervenção barata ao nível dos dados.

5.6 Robustez de perceção e composição de *augmentations*

Aplicámos a lógica da Secção 5.5 — sondar a política com transformações *held-out* — à perceção, distinguindo dois regimes de “perder cones”: (A) falha transiente, o *flicker* do sensor, em que a cada *frame* cada cone pode não ser detetado, reaparecendo no seguinte; e (B) cones fisicamente em falta, em que uma fração é removida *persistentemente* para toda a volta, com a *centerline* e a recompensa intactas. Os resultados são opostos: no regime transiente realista, até 15%, o *lap rate* mantém-se dentro do ruído amostral — a política média o ruído ao longo dos *steps*; já a remoção persistente colapsa o desempenho logo a 5–10% (a 10%: SAC 17%, PPO 10%, SAC+Mirror 30%). Não é *quantos* cones se perdem, é *por quanto tempo*: um cone em falta deixa um buraco de informação que uma política reativa, sem memória, não consegue preencher.

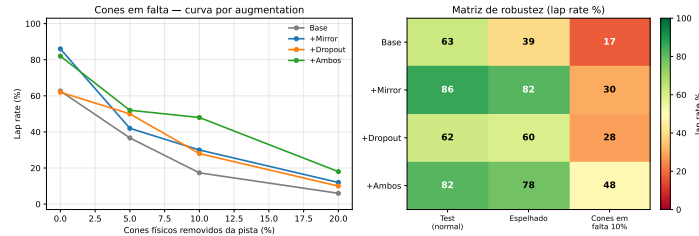


Fig. 3. Robustez por *augmentation*. Esquerda: *lap rate* vs. fração de cones fisicamente removidos. Direita: matriz de robustez (*lap rate* %) dos 4 modelos do *design* factorial nos três eixos de avaliação.

Esta fraqueza, tal como o viés direcional, é estrutural ao treino e corrige-se com *augmentation* dirigida. Avaliámos o *design* factorial completo — Base, +Mirror, +Dropout e +Ambos — nos três eixos — *test* normal, espelhado e cones em falta a 10% (Figura 3). Trata-se de uma re-avaliação independente, pelo que os valores absolutos diferem ligeiramente dos das tabelas anteriores, dentro do ruído amostral. Cada *augmentation* corrige sobretudo o seu eixo: +Mirror resolve o espelhado (39%→82%), +Dropout melhora os cones em falta (17%→28%); e a combinação é super-aditiva nos cones em falta, com 48%, acima dos 30% e 28% individuais, mantendo altos o espelhado, a 78%, e o *test* normal, a 82% — robustez nos dois eixos *sem custo no desempenho base*. Ressalvas: os modelos aumentados têm 1 *seed* cada, e nem a combinação salva o regime catastrófico, acima de 30% de remoção, cujo caminho natural seria dotar a política de memória.

Trabalho exploratório: PacSim e transferência sim-to-sim. Em paralelo, explorámos a integração com o PacSim, o simulador 3D em ROS2 usado na Formula Student, construindo uma *bridge* que liga a percepção de cones do PacSim à observação do agente e publica as ações como torque/*steering*. A exploração validou a viabilidade do acoplamento — a política 2D produz comandos coerentes no 3D — e a transferência completa fica como trabalho futuro, dependente de absorver o *gap* dinâmico de *pitch/roll* e *slip* de pneus.

6 Conclusões e Trabalho Futuro

Desenvolvemos um agente de RL capaz de conduzir um carro de Formula Student *Driverless* em pistas reais de competição, usando apenas a sua percepção local. As conclusões principais são: (i) o SAC e o PPO atingem *lap rates* semelhantes, mas o SAC é cerca de duas vezes mais seguro, o que o torna preferível num contexto onde cada cone é penalizado; (ii) o SAC é marcadamente mais eficiente em amostras, com 3M contra 6M *steps*; (iii) os *splits* de teste convencionais *mas-caram* um viés direcional estrutural de ~25 pp, que só uma avaliação em pistas espelhadas expõe; (iv) a robustez de percepção é assimétrica — o *flicker* transiente

é tolerado, mas cones fisicamente em falta colapsam a política; e (v) ambas as fraquezas se corrigem com *data augmentation* dirigida, cuja combinação é super-aditiva e sem custo no desempenho base — evidência de que eram limitações do protocolo de treino, não do algoritmo.

Como trabalho futuro destaca-se, em primeiro lugar, a evolução do ambiente para um modelo dinâmico de quatro rodas com pneus — aproximando a simulação 2D da dinâmica do PacSim e abrindo caminho à conclusão da transferência *sim-to-sim* iniciada na Secção 5.6; seguem-se a avaliação multi-volta e políticas com memória, como *frame-stacking* ou redes recorrentes, para mitigar a observabilidade parcial.

Glossário de siglas. DOO — *Down-Or-Out*; FOV — *Field of View*; FS-AI — *Formula Student Artificial Intelligence*; MLP — *Multi-Layer Perceptron*; MPC — *Model Predictive Control*; PPO — *Proximal Policy Optimization*; RL — *Reinforcement Learning*; SAC — *Soft Actor-Critic*; SB3 — *Stable-Baselines3*; SLAM — *Simultaneous Localization and Mapping*.

References

1. Kabzan, J., et al.: AMZ Driverless: The full autonomous racing system. *Journal of Field Robotics* **37**(7), 1267–1294 (2020). <https://doi.org/10.1002/rob.21977>
2. Sutton, R.S., Barto, A.G.: *Reinforcement Learning: An Introduction*. 2nd edn. MIT Press, Cambridge, MA (2018). ISBN 978-0-262-03924-6
3. Kong, J., Pfeiffer, M., Schildbach, G., Borrelli, F.: Kinematic and dynamic vehicle models for autonomous driving control design. In: 2015 IEEE Intelligent Vehicles Symposium (IV), pp. 1094–1099. IEEE (2015). <https://doi.org/10.1109/IVS.2015.7225830>
4. Haarnoja, T., et al.: Soft Actor-Critic algorithms and applications. arXiv:1812.05905 (2018). <https://arxiv.org/abs/1812.05905>
5. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal Policy Optimization algorithms. arXiv:1707.06347 (2017). <https://arxiv.org/abs/1707.06347>
6. Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., Dormann, N.: Stable-Baselines3: reliable reinforcement learning implementations. *Journal of Machine Learning Research* **22**(268), 1–8 (2021). <https://jmlr.org/papers/v22/20-1364.html>
7. Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., Abbeel, P.: Domain randomization for transferring deep neural networks from simulation to the real world. In: 2017 IEEE/RSJ IROS, pp. 23–30 (2017). <https://doi.org/10.1109/IROS.2017.8202133>
8. Ulrich, F., Wehrli, T.: End-to-End deep reinforcement learning for autonomous racing dynamics. Bachelor Thesis, ZHAW School of Engineering (2024). <https://doi.org/10.21256/zhaw-32000>